

Model Preparation (Machine Learning Part)

The program begins by preparing a machine learning model using the famous Iris dataset.

First, it loads the dataset using `load_iris()`, which contains measurements of flowers such as sepal length, sepal width, petal length, and petal width. These measurements act as **input features (X)**, while the flower categories (setosa, versicolor, virginica) are the **target labels (y)**.

Next, the dataset is split into two parts using `train_test_split()`:

- **Training data (70%)** → used to train the model
- **Testing data (30%)** → used to evaluate performance

A **Decision Tree Classifier** is then created and trained using the training data. This algorithm learns patterns from the flower measurements to classify them into the correct species.

After training, the model is saved to a file (`iris_model.pkl`) using `joblib`. This step is important because it allows the program to reuse the trained model later without retraining it every time the application runs.

GUI Application (User Interface Part)

After the model is prepared, the program builds a graphical user interface (GUI) using Tkinter.

The GUI window is created with a fixed size and includes:

- A title: *“Iris Flower Classification”*
- Labels for each input field
- Four entry boxes for user input:
 - Sepal Length
 - Sepal Width
 - Petal Length
 - Petal Width

A button labeled **“Classify Flower”** is also added.

User Interaction & Prediction

When the user clicks the button:

1. The program reads the values entered in the input boxes.

2. It converts these values from text to floating-point numbers.
 3. These values are passed to the loaded machine learning model.
 4. The model predicts the flower category (0, 1, or 2).
 5. This numeric result is converted into the actual flower name using the dataset's target names.
 6. The prediction is displayed in a popup message box.
-

Error Handling

The program includes basic error handling:

- If the user enters invalid (non-numeric) values, a `ValueError` occurs.
 - This is caught using a `try-except` block.
 - An error message is shown instead of crashing the program.
-

Program Flow

1. Check if the trained model file exists
2. If not, train and save the model
3. Load the saved model
4. Launch the GUI
5. Take user input → predict → display result

```
pip show numexpr
```

```
Name: numexpr
Version: 2.10.1
Summary: Fast numerical expression evaluator for NumPy
Home-page: https://github.com/pydata/numexpr
Author:
Author-email: "David M. Cooke, Francesc Alted, and others"
<blosc@blosc.org>
License:
Location: C:\ProgramData\anaconda3\Lib\site-packages
Requires: numpy
Required-by: tables
Note: you may need to restart the kernel to use updated packages.
```

```
# =====
# Iris Flower Classification using Decision Tree
# =====
```

```
# -----
# LIBRARIES USED
# -----
```

```
import os
# os = Operating System module
# Used for file handling operations.
# Here used to check whether model file exists or not.
```

```
import joblib
# joblib is used to save and load machine learning models.
# It stores trained model into a file.
```

```
import tkinter as tk
# tkinter is Python's built-in GUI library.
# Used to create windows, labels, buttons, textboxes etc.
```

```
from tkinter import messagebox
# messagebox creates popup windows.
# Used for success message and error message.
```

```
from sklearn.datasets import load_iris
# load_iris() loads famous Iris dataset.
# Dataset has flower measurements.
```

```
from sklearn.model_selection import train_test_split
# Used to divide dataset into:
# 1. Training data
# 2. Testing data
```

```
from sklearn.tree import DecisionTreeClassifier
```

```

# Imports Decision Tree machine learning algorithm.
# Used for classification.

# -----
# FUNCTION 1 : BUILD MODEL
# -----

def build_model():
    # Function named build_model()
    # Purpose: Train machine learning model.

    iris = load_iris()
    # Loads iris flower dataset.

    X = iris.data
    # X = Input features
    # Contains 4 flower measurements:
    # Sepal Length
    # Sepal Width
    # Petal Length
    # Petal Width

    y = iris.target
    # y = Output labels
    # 0 = setosa
    # 1 = versicolor
    # 2 = virginica

    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.30,
        random_state=1
    )
    # Splits dataset:
    # 70% Training data
    # 30% Testing data
    # random_state=1 gives same split every run

    model = DecisionTreeClassifier()
    # Creates Decision Tree model.

    model.fit(X_train, y_train)
    # Trains model using training data.

    joblib.dump(model, "iris_model.pkl")
    # Saves trained model in file:
    # iris_model.pkl

```

```

# -----
# FUNCTION 2 : CREATE GUI
# -----

def run_gui():
    # Function to create graphical window.

    model = joblib.load("iris_model.pkl")
    # Loads saved model.

    iris = load_iris()
    # Loads dataset again.
    # Used to get flower names.

# -----
# BUTTON FUNCTION
# -----

def classify_flower():
    # Runs when button clicked.

    try:
        # Try block used to catch invalid input.

        features = [[
            float(entry1.get()),
            float(entry2.get()),
            float(entry3.get()),
            float(entry4.get())
        ]]
        # Reads values from entry boxes.
        # Converts text into decimal numbers.
        # Example:
        # [[5.1, 3.5, 1.4, 0.2]]

        prediction = model.predict(features)[0]
        # Predict flower class.
        # Returns 0 or 1 or 2

        flower_name = iris.target_names[prediction]
        # Converts numeric result into name.
        # Example:
        # 0 = setosa

        messagebox.showinfo(
            "Prediction Result",
            f"The Iris flower is: {flower_name}"
        )
        # Popup window showing prediction.

```

```

except ValueError:
    # If user enters wrong text like abc

    messagebox.showerror(
        "Input Error",
        "Please enter valid numeric values."
    )
    # Shows error popup.

# -----
# MAIN WINDOW
# -----

root = tk.Tk()
# Creates main GUI window.

root.title("Iris Flower Classification")
# Sets window title.

root.geometry("400x320")
# Sets window size:
# Width = 400 pixels
# Height = 320 pixels

root.resizable(False, False)
# Prevent resizing of window.

# -----
# HEADING LABEL
# -----

tk.Label(
    root,
    text="Iris Flower Prediction",
    font=("Arial", 14, "bold")
).grid(row=0, column=0, columnspan=2, pady=10)

# Creates heading text.
# row=0 means first row
# columnspan=2 means covers 2 columns

# -----
# LABELS
# -----

tk.Label(root, text="Sepal Length (cm):").grid(row=1, column=0,
padx=10, pady=8)

```

```

    tk.Label(root, text="Sepal Width (cm):").grid(row=2, column=0,
padx=10, pady=8)

    tk.Label(root, text="Petal Length (cm):").grid(row=3, column=0,
padx=10, pady=8)

    tk.Label(root, text="Petal Width (cm):").grid(row=4, column=0,
padx=10, pady=8)

    # These lines create text labels.

# -----
# ENTRY BOXES
# -----

    entry1 = tk.Entry(root)
    # Input box for Sepal Length

    entry2 = tk.Entry(root)
    # Input box for Sepal Width

    entry3 = tk.Entry(root)
    # Input box for Petal Length

    entry4 = tk.Entry(root)
    # Input box for Petal Width

# -----
# POSITION ENTRY BOXES
# -----

    entry1.grid(row=1, column=1, padx=10, pady=8)
    entry2.grid(row=2, column=1, padx=10, pady=8)
    entry3.grid(row=3, column=1, padx=10, pady=8)
    entry4.grid(row=4, column=1, padx=10, pady=8)

    # Places entry boxes beside labels.

# -----
# BUTTON
# -----

    tk.Button(
        root,
        text="Classify Flower",
        width=18,
        bg="lightblue",

```

```

        command=classify_flower
    ).grid(row=5, column=0, columnspan=2, pady=20)

    # Creates button.
    # command=classify_flower means:
    # when clicked run prediction.

# -----
# RUN GUI
# -----

    root.mainloop()
    # Keeps window open.
    # Waits for user actions.

# -----
# MAIN PROGRAM
# -----

if __name__ == "__main__":
    # Runs only when file is executed directly.

    if not os.path.exists("iris_model.pkl"):
        # Checks if model file exists.

        build_model()
        # If file missing:
        # train model and save file.

    run_gui()
    # Opens GUI window.

```


Iris Flower Classification

Iris Flower Prediction

Sepal Length (cm):

Sepal Width (cm):

Petal Length (cm):

Petal Width (cm):

Prediction Result

 The Iris flower is: virginica

Iris Flower Classification

Iris Flower Prediction

Sepal Length (cm):

Sepal Width (cm):

Petal Length (cm):

Petal Width (cm):

Prediction Result

 The Iris flower is: versicolor